

— OPENAI SYMPHONY · A COMPLETE GUIDE

Symphony × Claude Code – the **Engine**, the **Target**, and the loop between them

A long-running orchestrator watches a **Linear board** for actionable tickets and, per ticket, spawns an autonomous **Claude Code** agent in an isolated workspace — driving it turn by turn to Done. Design lives in git as **aidlc-docs**; execution lives in Linear as tickets and the **## Workpad**.

2 repos · engine + target

Linear = the board

Claude Code = the agent

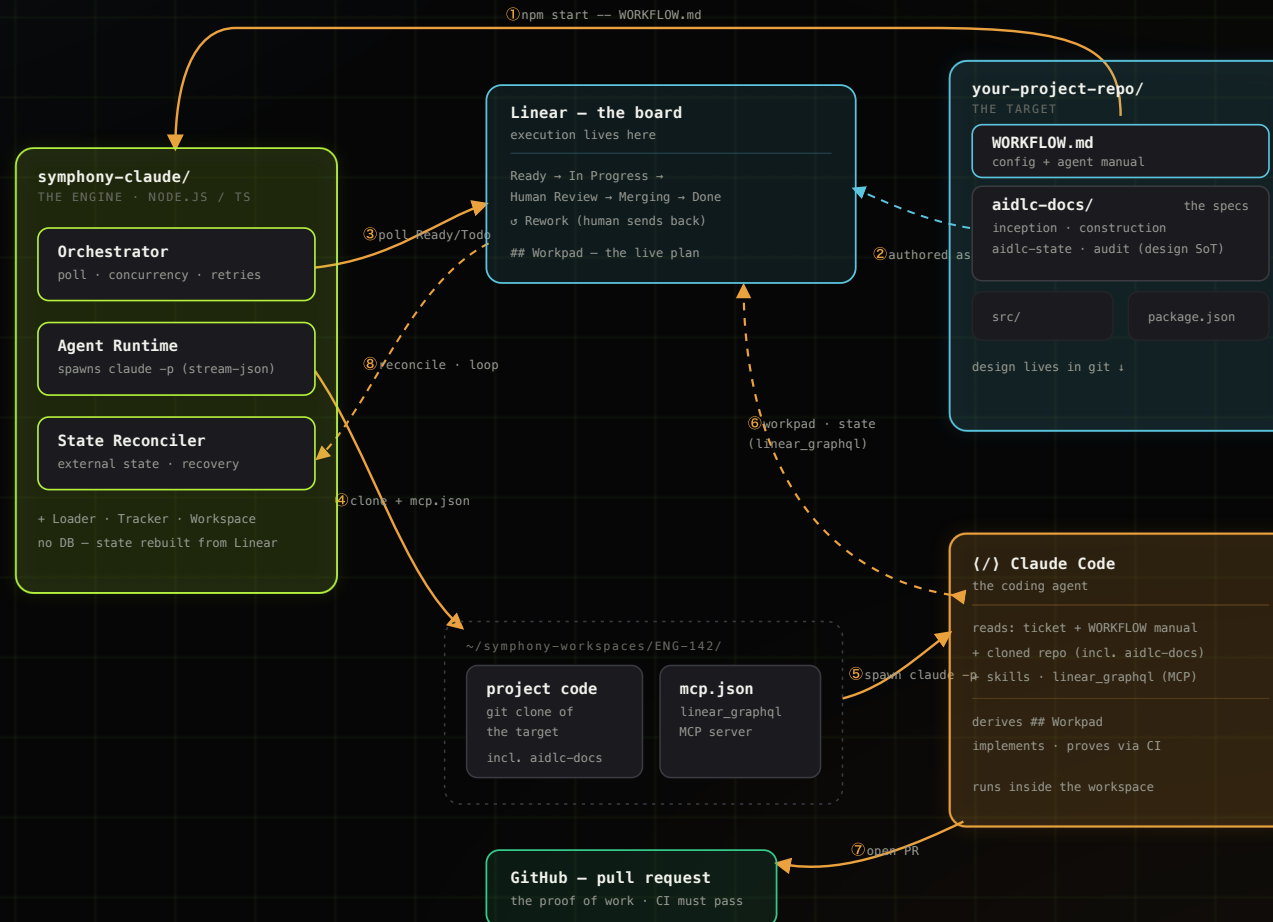
claude -p · stream-json

no DB · state rebuilt from Linear

— THE WHOLE MAP

Every moving part, and the loop that connects them

□ The Engine (symphony-claude)
 □ The Target & Linear
 □ Agent runtime (Claude Code)
 □ Proof / done
 — happy path ①→⑧
 - - - authored / read-write / loop



— THE CAST

Two repos, a board, and a runtime

Every part has one home. The **engine** runs the loop, the **target** holds the code and its design, and **Linear** is where the work is tracked.

symphony-claude/

THE ENGINE · NODE.JS / TS

Orchestrator

poll · concurrency · retries

Agent Runtime

spawns `claude -p (stream-json)`

State Reconciler

external state · recovery

+ Loader · Tracker · Workspace
no DB — state rebuilt from Linear

your-project-repo/

THE TARGET

WORKFLOW.md

config + agent manual

aidlc-docs/

inception · construction · aidlc-state · audit — the design source of truth

src/ · package.json

the actual project code

design lives in git

Linear – the board

EXECUTION LIVES HERE

Ready → In Progress →
Human Review → Merging →
Done

↺ Rework (human sends back)

Workpad – the live plan

a ticket is the unit of work

THE NARRATIVE

End to end, in eight steps

01-02 · SETUP & PLANNING

03-08 · THE AUTONOMOUS LOOP, PER TICKET

01

Boot

`npm start -- WORKFLOW.md` points the engine at the target's config + agent manual.

02

Plan

During inception/construction, you + Claude Code write `aidlc-docs/` and author Linear tickets in Todo.

03

Poll

The Orchestrator polls Linear for tickets in the configured active states.

04

Isolate

Agent Runtime creates `~/symphony-workspaces/ENG-142/`, git-clones the target, writes `mcp.json`.

05

Spawn

Launches `claude -p --output-format stream-json --mcp-config` inside the workspace.

06

Work

Claude reads the ticket + repo (incl. `aidlc-docs`), derives the `## Workpad`, implements, moves the state.

07

Prove

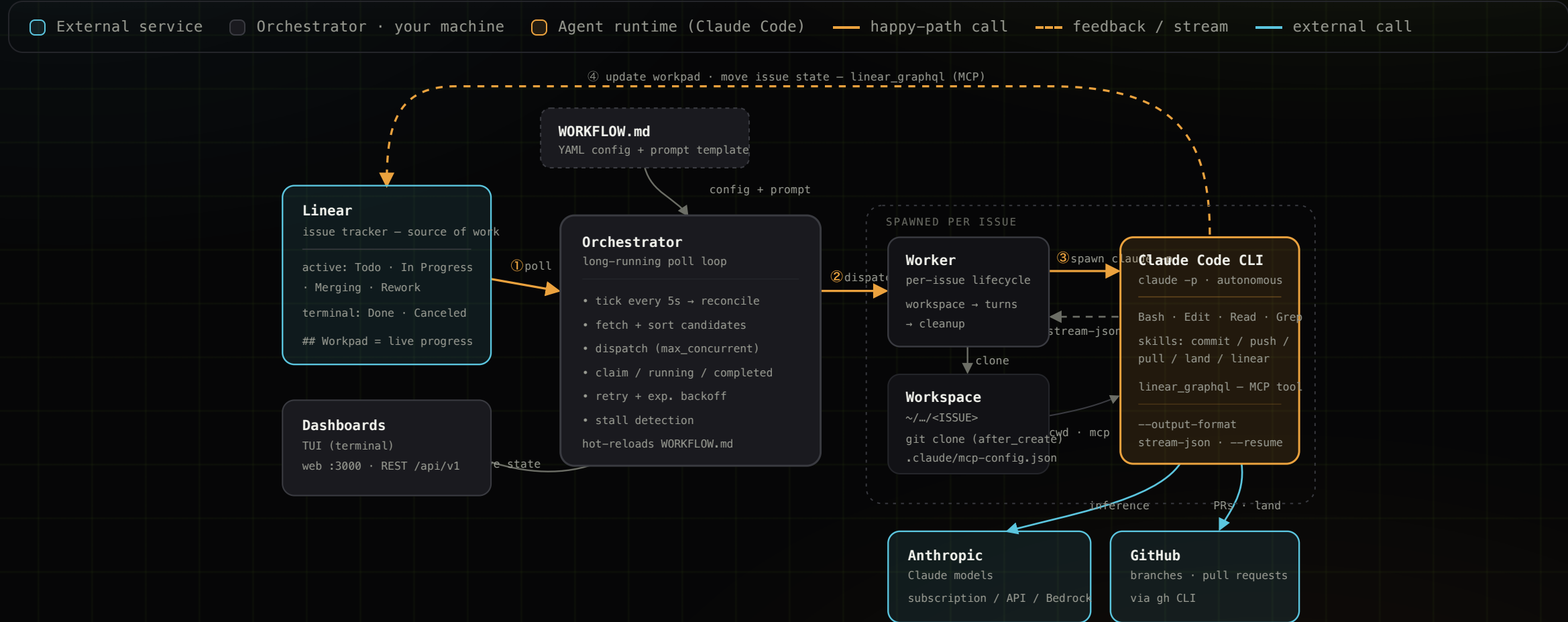
Opens a PR — CI is the proof of work — attaches it to the ticket, moves it to Human Review.

08

Reconcile

The State Reconciler watches Linear; the loop resumes or stops when the ticket reaches Done.

Orchestrator → Worker → Claude Code CLI

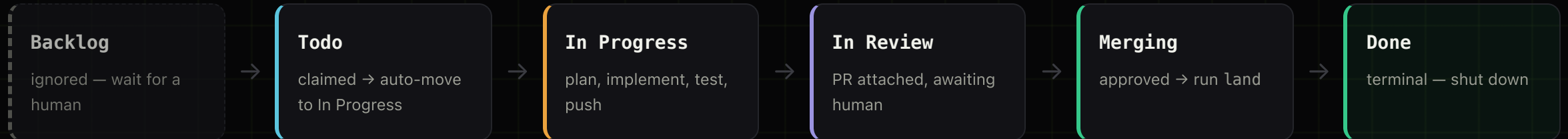


Happy path ①→④ in amber. The orchestrator never talks to Claude's model directly – it drives the Claude subprocess and reads its stream-json events; the agent reaches Linear/GitHub/Anthropic on its own.

STATUS MAP

How a ticket moves

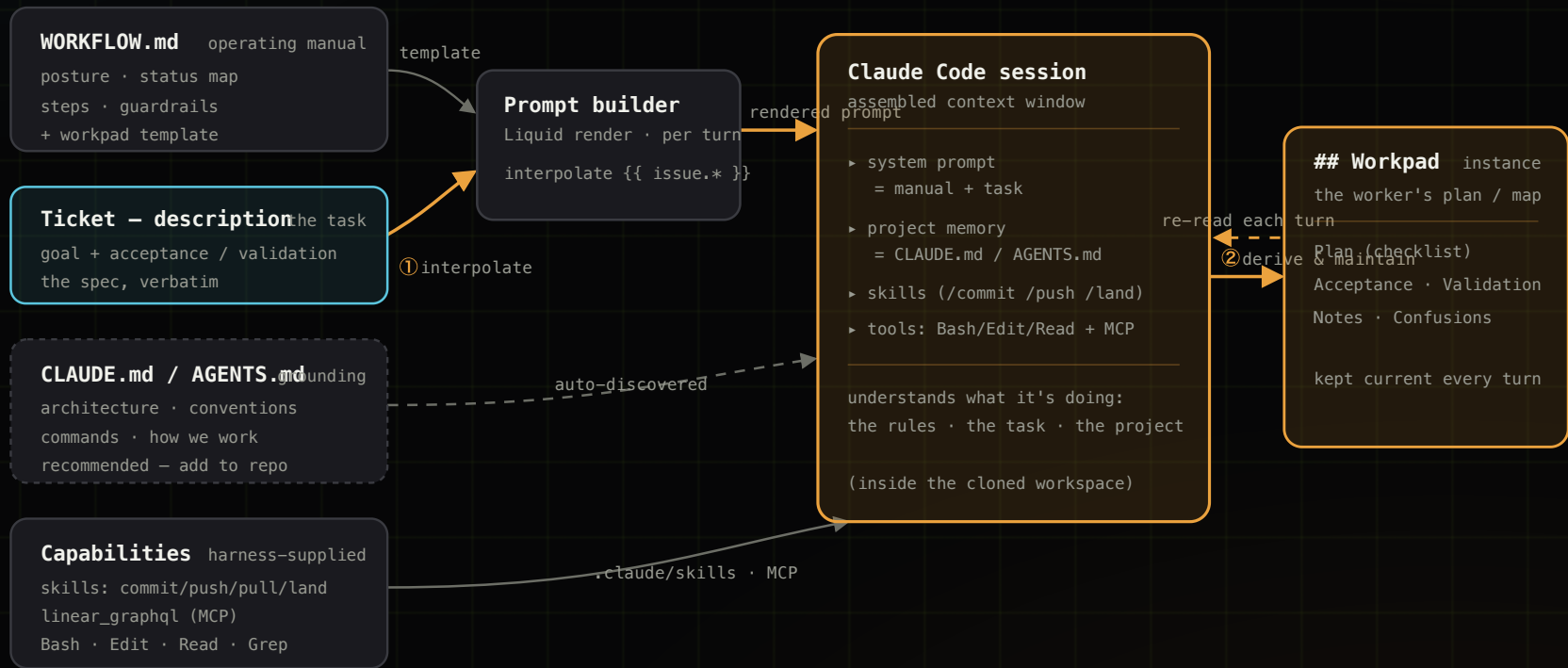
The Linear state is the program counter. The agent reads it to decide what to do, and writes it to hand off — to itself on the next turn, or to a human for review.



Rework is a branch off In Review: a reviewer requests changes, so the agent closes the PR, wipes the workpad, cuts a fresh branch from origin/main, and starts the loop over — a full reset, not a patch.

How the worker gets its bearings

□ The task (ticket description)
□ Harness-supplied context
□ Agent + its workpad instance
□ Recommended – add if missing



① The prompt builder interpolates the ticket description into the WORKFLOW.md template. Claude Code then auto-loads project memory (CLAUDE.md / AGENTS.md), skills, and linear_graphql from the cloned workspace. ② From that assembled context the worker maintains the ## Workpad – re-read and reconciled every turn.

THE MAPPING

How a spec becomes a ticket becomes execution

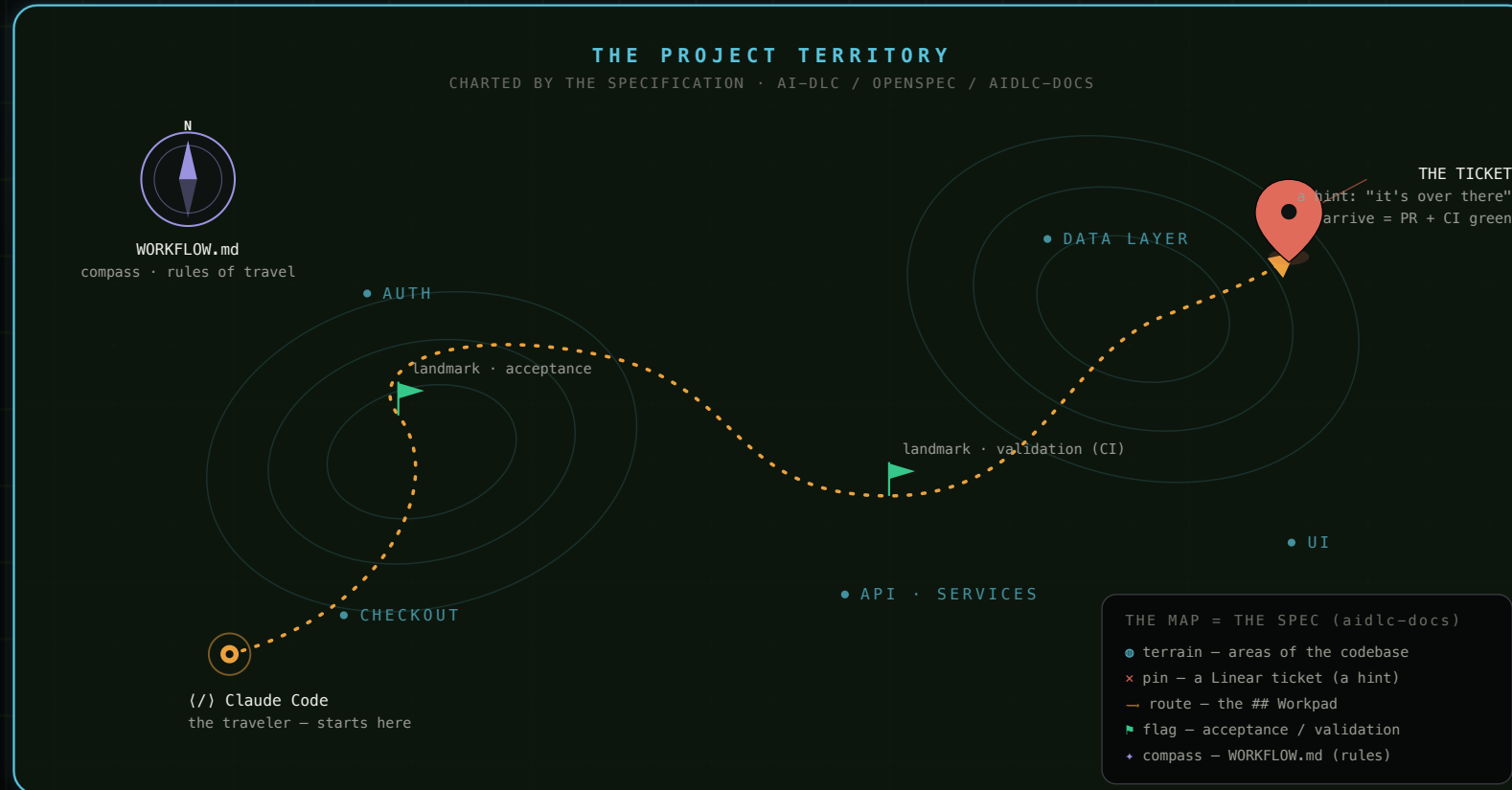
Each part of the spec lands in a specific ticket location, then gets projected into a specific workpad section — with a gate where one applies. This is the spec-driven contract the worker follows.

SPEC ELEMENT	WHERE ON THE TICKET	WORKPAD SECTION	GATE
Intent / goal	Description body	Plan	—
Acceptance criteria	Description / derived	Acceptance Criteria	all checked
Test plan / validation	Description Validation · Test Plan · Testing	Validation	green + no downgrade
Scope (one unit)	one ticket	whole workpad	one PR
Dependencies	blockers / blockedBy	Plan notes	—
Stage / status	state field	(drives routing)	quality bar per state
Progress / handoff	—	Notes + checklist	single workpad only
Result link	PR attachment	—	symphony label

— THE ONE RULE THAT ANCHORS IT ALL

The spec is the **map**; the ticket is a **pin**.

— Design in git, execution in Linear.



The whole frame is the **map** – the **spec**, which lives in **git** (design: aidlc-docs/). The **ticket** is the smallest thing on it: a pin, tracked in **Linear** (execution). The worker (**Claude Code**) reads the map, orients by the compass (**WORKFLOW.md**), and traces a **route** (the **## Workpad**) to the pin – confirming **acceptance & validation** landmarks. Never invert it: **design in git, state in Linear**.